Indice

1 Maps **2**
1.1 Map Abstract Data Type 2
1.1.1 Esempio di Utilizzo 3
1.2 User-defined Map - Mutable Mapping ABC della Collection Module 4
1.3 Sorted Maps - Array Based e Binary Search Tree 4
1.3.1 Array Based Sorted Map 5
1.3.2 Binary Search Tree Sorted Map 5
1.3.2.1 Predecessor & Successor 6
1.3.2.2 Search 7
1.3.2.3 Insert 8
1.3.2.4 Delete 9
1.3.2.5 Performance dei metodi PSSID 10

Maps

Appunti di Programmazione

25 marzo 2026

1 Maps

Maps and Dictionaries

Python's **dict** class is arguably the most significant data structure in the language. It represents an abstraction known as a ***dictionary*** in which unique ***keys*** are mapped to associated ***values***. Because of the relationship they express between keys and values, dictionaries are commonly known as ***associative arrays*** or ***maps***. In this book, we use the term *dictionary* when specifically discussing Python's dict class, and the term *map* when discussing the more general notion of the abstract data type.

As a simple example, Figure 10.1 illustrates a map from the names of countries to their associated units of currency.

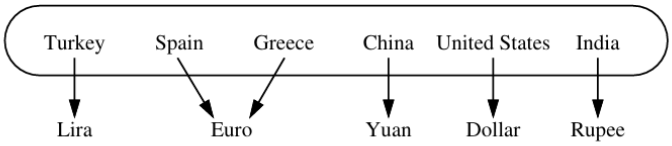


Figure 10.1: A map from countries (the keys) to their units of currency (the values).

We note that the keys (the country names) are assumed to be unique, but the values (the currency units) are not necessarily unique. For example, we note that Spain and Greece both use the euro for currency. Maps use an array-like syntax for indexing, such as currency['Greece'] to access a value associated with a given key or currency['Greece'] = 'Drachma' to remap it to a new value. Unlike a standard array, indices for a map need not be consecutive nor even numeric. Common applications of maps include the following.

Figura 1: Maps definition

1.1 Map Abstract Data Type

Queste sono le operazioni che una Map class dovrebbe supportare:

10.1.1 The Map ADT

In this section, we introduce the ***map ADT***, and define its behaviors to be consistent with those of Python's built-in dict class. We begin by listing what we consider the most significant five behaviors of a map *M* as follows:

M[k]: Return the value v associated with key k in map M, if one exists; otherwise raise a KeyError. In Python, this is implemented with the special method __getitem__.

M[k] = v: Associate value v with key k in map M, replacing the existing value if the map already contains an item with key equal to k. In Python, this is implemented with the special method __setitem__.

del M[k]: Remove from map M the item with key equal to k; if M has no such item, then raise a KeyError. In Python, this is implemented with the special method __delitem__.

len(M): Return the number of items in map M. In Python, this is implemented with the special method __len__.

iter(M): The default iteration for a map generates a sequence of *keys* in the map. In Python, this is implemented with the special method __iter__, and it allows loops of the form, **for** k **in** M.

k in M: Return True if the map contains an item with key k. In Python, this is implemented with the special __contains__ method.

M.get(k, d=None): Return M[k] if key k exists in the map; otherwise return default value d. This provides a form to query M[k] without risk of a KeyError.

M.setdefault(k, d): If key k exists in the map, simply return M[k]; if key k does not exist, set M[k] = d and return that value.

M.pop(k, d=None): Remove the item associated with key k from the map and return its associated value v. If key k is not in the map, return default value d (or raise KeyError if parameter d is None).

Chapter 10. Maps, Hash Tables, and Skip Lists

M.popitem(): Remove an arbitrary key-value pair from the map, and return a (k,v) tuple representing the removed pair. If map is empty, raise a KeyError.

M.clear(): Remove all key-value pairs from the map.

M.keys(): Return a set-like view of all keys of M.

M.values(): Return a set-like view of all values of M.

M.items(): Return a set-like view of (k,v) tuples for all entries of M.

M.update(M2): Assign M[k] = v for every (k,v) pair in map M2.

M == M2: Return True if maps M and M2 have identical key-value associations.

M != M2: Return True if maps M and M2 do not have identical key-value associations.

Figura 2: Map ADT

1.1.1 Esempio di Utilizzo

Operation	Return Value	Map
len(M)	0	{ }
M['K'] = 2	–	{'K': 2}
M['B'] = 4	–	{'K': 2, 'B': 4}
M['U'] = 2	–	{'K': 2, 'B': 4, 'U': 2}
M['V'] = 8	–	{'K': 2, 'B': 4, 'U': 2, 'V': 8}
M['K'] = 9	–	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M['B']	4	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M['X']	KeyError	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('F')	None	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('F', 5)	5	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
M.get('K', 5)	9	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
len(M)	4	{'K': 9, 'B': 4, 'U': 2, 'V': 8}
del M['V']	–	{'K': 9, 'B': 4, 'U': 2}
M.pop('K')	9	{'B': 4, 'U': 2}
M.keys()	'B', 'U'	{'B': 4, 'U': 2}
M.values()	4, 2	{'B': 4, 'U': 2}
M.items()	('B', 4), ('U', 2)	{'B': 4, 'U': 2}
M.setdefault('B', 1)	4	{'B': 4, 'U': 2}
M.setdefault('A', 1)	1	{'A': 1, 'B': 4, 'U': 2}
M.popitem()	('B', 4)	{'A': 1, 'U': 2}

Figura 3: Esempio di funzionamento mapADT

1.2 User-defined Map - Mutable Mapping ABC della Collection Module

Per costruire le classe Maps costruiamo **MapBase** che eredità dalla Abstract Base Classe **MutableMapping**. MutableMapping è del modulo *Collections*.

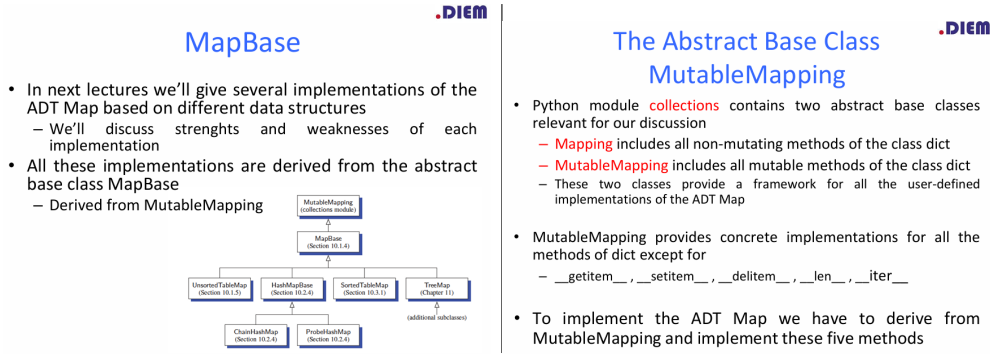


Figura 4: Map Base e Mutable Mapping ABC

1.3 Sorted Maps - Array Based e Binary Search Tree

Sorted Map è una Mappa **Ordinata**, quindi è possibile definire un ordine tra le chiavi. Sorted Map eredita da MapBase.

L'Abstract Data Type di una Sorted Map è la seguente:

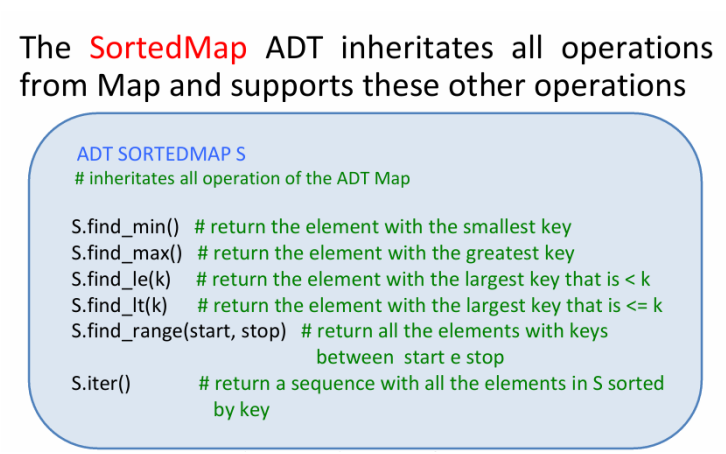


Figura 5: Sorted Map ADT

1.3.1 Array Based Sorted Map

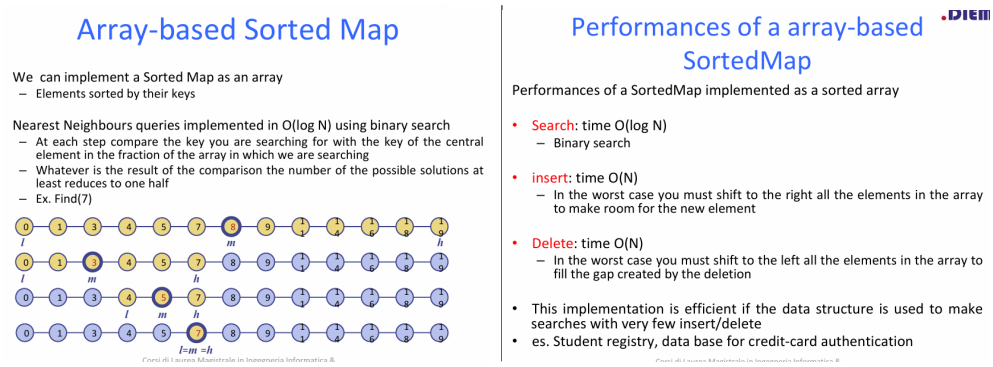


Figura 6: Array Based Sorted Map

1.3.2 Binary Search Tree Sorted Map

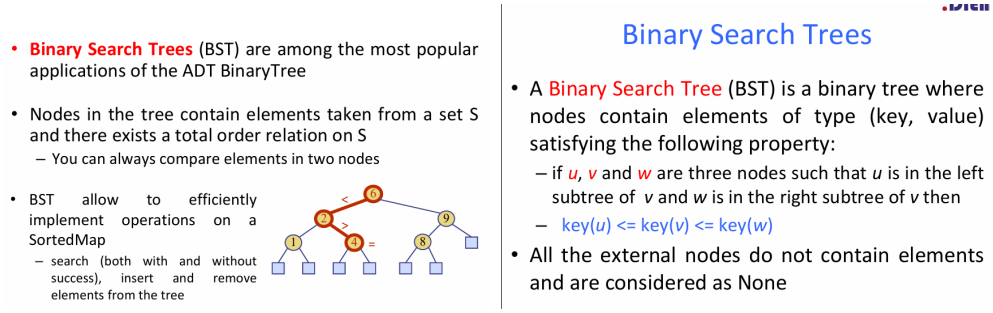


Figura 7: Binary Search Tree Sorted Map

1.3.2.1 Predecessor & Successor Pred e Succ

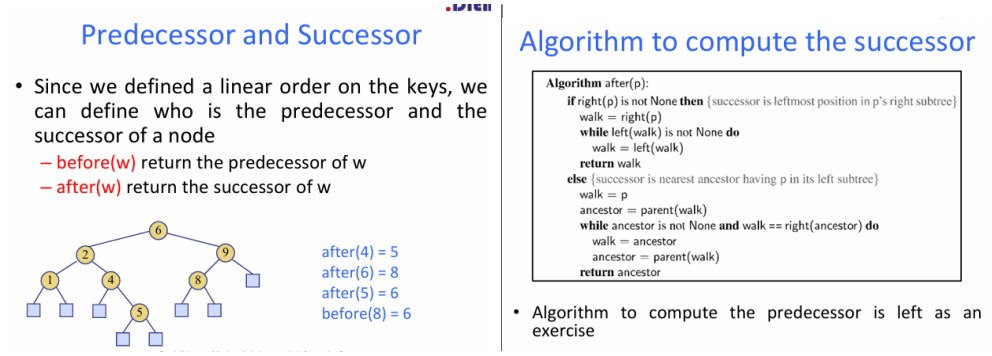


Figura 8: Predecessor & Successor

1.3.2.2 **Search** Search

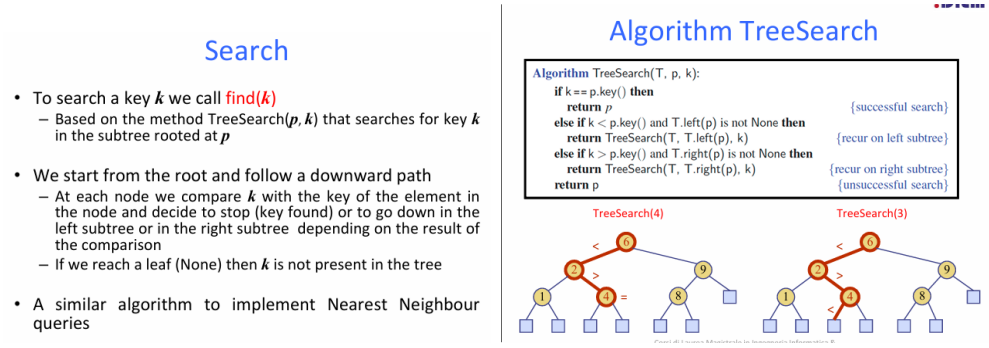


Figura 9: Search

1.3.2.3 **Insert** Insert

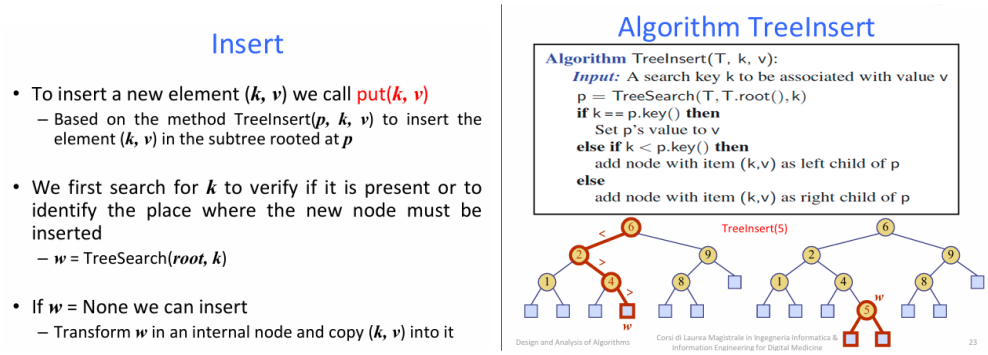


Figura 10: Insert

1.3.2.4 **Delete** Delete

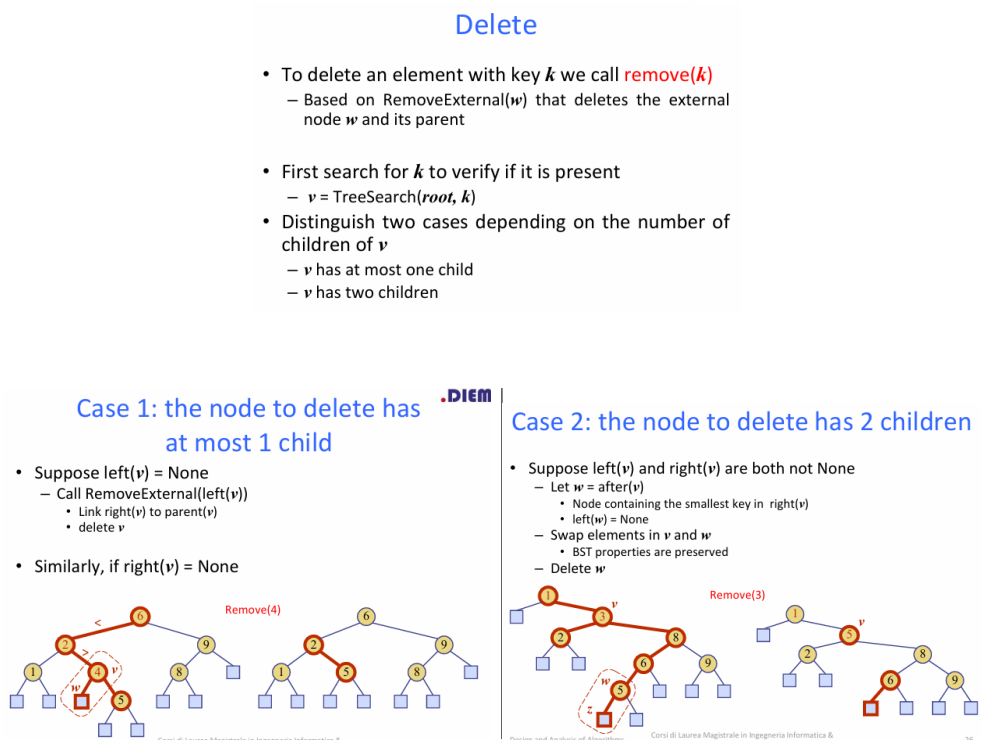
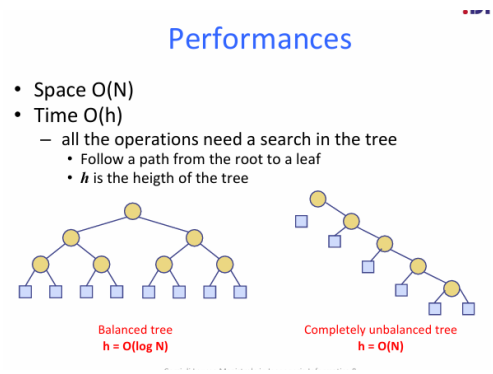


Figura 11: Delete

1.3.2.5 Performance dei metodi PSSID Performances



Chapter 11. Search Trees

Operation	Running Time
k in T	$O(h)$
T[k], T[k] = v	$O(h)$
T.delete(p), del T[k]	$O(h)$
T.find_position(k)	$O(h)$
T.first(), T.last(), T.find_min(), T.find_max()	$O(h)$
T.before(p), T.after(p)	$O(h)$
T.find_lt(k), T.find_le(k), T.find_gt(k), T.find_ge(k)	$O(h)$
T.find_range(start, stop)	$O(s+h)$
iter(T), reversed(T)	$O(n)$

Table 11.1: Worst-case running times of the operations for a TreeMap T. We denote the current height of the tree with h, and the number of items reported by find_range as s. The space usage is $O(n)$, where n is the number of items stored in the map.

A binary search tree T is therefore an efficient implementation of a map with n entries only if its height is small. In the best case, T has height $h = \lceil \log(n+1) \rceil - 1$, which yields logarithmic-time performance for all the map operations. In the worst case, however, T has height n, in which case it would look and feel like an ordered list implementation of a map. Such a worst-case configuration arises, for example, if we insert items with keys in increasing or decreasing order. (See Figure 11.7.)

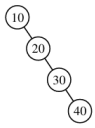


Figure 11.7: Example of a binary search tree with linear height, obtained by inserting entries with keys in increasing order.

Figura 12: Performance BST Methods